

Project Interim Report
On
File Compression Simulator
(Using Core Java & OOP Concepts)

RU-100-01-00012
Object Oriented Programming With Java

SESSION: 2025-26



Submitted By:
Rohit Kumar
Gope

RU-25-11202
Bachelor of Technology in Computer Science &
Engineering with AI and ML in association with
Google

Under the Guidance of
Mr. Ashish Shivare

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI , CG

(April 2026)

Project Title: File Compression Simulator

(Using Core Java & OOP Concepts)

1. Problem Statement

With the exponential growth of digital data in modern computing environments, efficient storage and data transmission have become critical concerns. Large files consume significant disk space and slow down network transfers.

The objective of this project is to design and implement a **File Compression Simulator** using Object-Oriented Programming principles in Core Java. The system should simulate compression and decompression of files using algorithms such as Run-Length Encoding (RLE), calculate compression ratios, display file size before and after compression, and provide a menu-driven interface for user interaction.

2. Objective

- To model a real-world file compression and decompression system.
- To implement OOP concepts like classes, objects, inheritance, and methods.
- To manage relationships between entities like files, compression algorithms, and output results.
- To perform compression ratio and space-saving calculations.

3. System Description

The system will consist of three main components:

A. FileData Class

Represents a file to be compressed.

Attributes:

- String fileName
- String originalContent
- int originalSize

Responsibilities:

- Store file name and original content
- Provide access to file data and size

B. CompressionAlgorithm Class

Represents a compression technique applied to the file.

Attributes:

- String algorithmName
- String compressedContent
- double compressionRatio

Responsibilities:

- Apply compression logic to file data
- Return compressed content and ratio

C. CompressionResult Class

Represents the result of a compression operation.

Attributes:

- FileData originalFile
- String compressedData
- int compressedSize
- double spaceSaved
- double compressionRatio

Responsibilities:

- Hold compression results
- Calculate and display space saved
- Generate compression summary report

4. Functional Requirements

Your system should:

1. Allow users to input or load file content for compression.
2. Allow users to: ■ Browse available compression algorithms ■ Select a compression method ■ View original and compressed file sizes
3. Calculate: ■ Compressed size (characters after encoding) ■ Space saved (original size – compressed size) ■ Compression ratio ■ Decompressed output
4. Display complete compression summary report.

5. Approach / Implementation Steps

Step 1: Create FileData Class

- Define:
 - fileName, originalContent, originalSize
- Create constructor and getter methods

Step 2: Create CompressionAlgorithm Class

- Store:
 - Algorithm name
 - Compression logic (e.g., RLE)
- Method to apply compression on given content

Step 3: Create CompressionResult Class

- Maintain:
 - Original file reference
 - Compressed data string
 - Size metrics

- Methods:

```
void compress(FileData file)
void decompress()
void displaySummary()
```

Step 4: Implement Compression Logic

RLE Compression (Run-Length Encoding):

Example: "AAABBBCC" → "3A3B2C"

Compression Ratio Calculation:

```
compressionRatio = compressedSize / originalSize
```

Space Saved Calculation:

```
spaceSaved = originalSize - compressedSize
```

```
percentageSaved = (spaceSaved / originalSize) * 100
```

Step 5: Display Compression Summary

Include:

- File name
- Original size (in characters/bytes)
- Compressed data
- Compressed size
- Space saved
- Compression ratio
- Decompressed output

6. Expected Output Example

```
Compression Summary:
```

```
-----
```

```
File Name : sample.txt
```

```
Original Data : AAABBBCCDDDD
```

```
Original Size : 12 characters
```

```
Algorithm Used : Run-Length Encoding (RLE)
```

```
Compressed Data : 3A3B2C4D
```

```
Compressed Size : 8 characters
```

```
Space Saved : 4 characters
```

```
Savings (%) : 33.33%
```

```
Compression Ratio: 0.67
```

```
Decompressed Data: AAABBBCCDDDD
```

```
Decompression Successful: YES
```

7. Constraints

- Must use OOP concepts properly.
- Use arrays (not ArrayList) unless specified.
- Implement clean method separation.
- Avoid hardcoding logic unnecessarily.

8. Bonus Features (Optional)

- Support for multiple compression algorithms (RLE, Huffman simulation)
- File size comparison with visual bar representation
- Batch compression of multiple files

- User input via Scanner for custom file content
- Compression history tracking

9. Evaluation / Grading Criteria (Total: 25 Marks)

A. Viva (8 Marks)

Criteria	Marks
Understanding of OOP concepts	3
Explanation of system design	2
Compression logic explanation	2
Confidence & communication	1

B. Report (7 Marks)

Criteria	Marks
Problem understanding	2
Proper class diagram / explanation	2
Compression logic clarity	2
Presentation & formatting	1

C. Code (10 Marks)

Criteria	Marks
Correct class structure	3
Proper use of arrays & methods	2
Accurate compression logic	3
Clean output formatting	1
Code readability & naming conventions	1

10. Submission Guidelines

- Submit:
 - Source code (.java files)
 - Project report (PDF/Word)
- Code should be well-commented.
- Follow proper naming conventions.